MONASH University
Engineering

# ECE2071 Project S1 2026 - Week 5 Activity (Malaysia Campus)

# Task 2. Interfacing the HC-SR04 Ultrasonic Distance Ranging Module to the Microcontroller and Transmission of Measured Distance using UART

### Synopsis

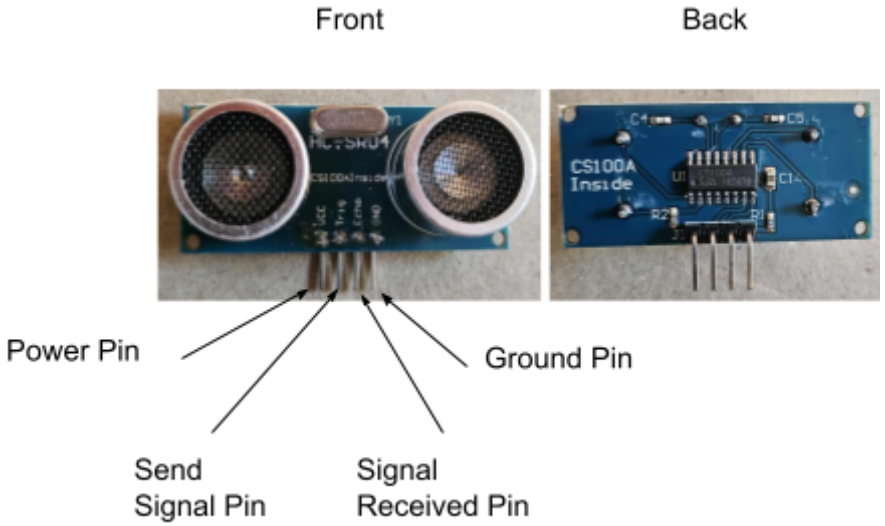
In this task, we will interface the HC-SR04 ultrasonic distance ranging module to the microcontroller utilizing timers with interrupts, and then transmit the distances measured by the ultrasonic sensor to the computer through serial communication using the universal asynchronous receiver transmitter (UART) interface on the STM32 Nucleo-L432KC board.

### Required Materials

- STM32 Nucleo-L432KC board
- USB Type A-to-Micro B cable
- Personal Computer installed with STM32 CubIDE software, Termite RS-232 terminal and Python Interpreter
- HC-SR04 ultrasonic distance ranging module
- Connecting wires

## Task 2(a): Interfacing the HC-SR04 ultrasonic sensor to the the microcontroller

In order to determine the distance from a user, as required in the project specification, an ultrasonic sensor can be used. The ultrasonic sensor provided in the ElecLab kit is the HC-SR04. This module has 4 pins: VCC, GND, Trig and Echo as shown in the image below.




The working principle of the ultrasonic sensor is shown in the timing diagram on the next page. Once the sensor is powered up through the **VCC and GND** pins, distance measurement is initiated by sending a 10 microsecond pulse to the **Trig** (Trigger) pin of the sensor. This will cause the sensor to send out a sonic burst of 8 cycles at a frequency of 40 kHz, and at the same time **Echo** pin is driven to a state of 'Logic 1'. Now, the sensor waits to receive the reflected ultrasonic waves from the nearest obstacle while maintaining the logic

level at the Echo pin. Further, once the reflected waves are detected by the sensor, the Echo pin returns back to the state of 'Logic 0'. The round-trip time taken by the ultrasonic waves to reach the nearest obstacle, reflect and then return back to the sensor can then be calculated based on the time during which the Echo pin remained at the state of 'Logic 1' as shown in the equation below.

$$Distance \ = \ (Echo\ high\ level\ time \times velocity\ of\ sound)/2$$

Be aware that in case a signal is not received, the Echo pin will remain at Logic 1 until a new trigger is sent, and thus the system needs to have a timeout set for no response received.

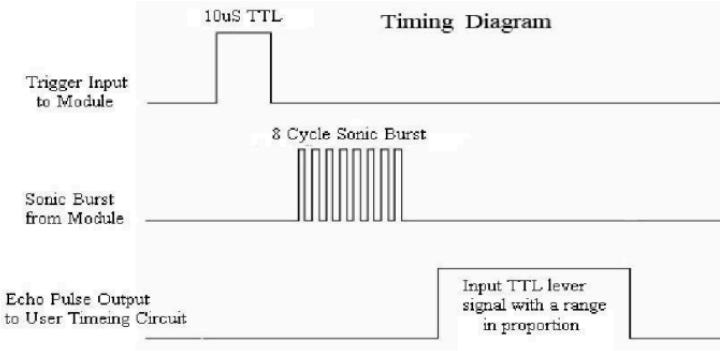
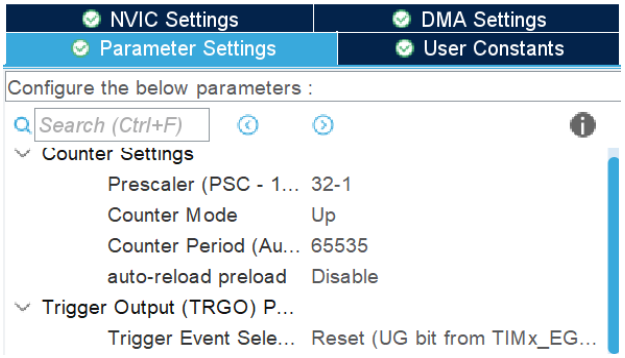



*Figure 44: HC-SR04 Ultrasonic module Signal Timing diagram from the data sheet* (https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf)

**Step 1:** Connect the HC-SR04 ultrasonic module to the STM32 Nucleo-L432KC board as follows:

| HC-SRO4 pin | STM32 Nucleo-L432KC board pin |
|---|---|
| Trig | Pin A5 (Pin PA6 on the microcontroller configured as GPIO output) |
| Echo | Pin D9 (Pin PA8 on the microcontroller configured as input pin to timer for capture ⟶ will be performed later in Step 5) |
| VCC | 5V |
| GND | GND |

**Step 2:** Create a new STM32 project on the STM32Cube IDE. Then, activate and configure timer TIM6 under the .ioc file of the project such that the timer counts up at a frequency of 1 MHz until the maximum value of 65535. The parameter settings for TIM6 should be chosen as shown in the image below (assuming APB1 clock frequency of 32 MHz). This timer will be utilized to set the Trig pin of the ultrasonic sensor to 'Logic 1' for a duration of 10 microseconds.




**Step 3:** Save the .ioc file and generate code. In the file named `main.c`, add the following code in between the **`/* USER CODE BEGIN 2 */`** and **`/* USER CODE END 2 */`** blocks to start the timer TIM6 under basic mode.

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start(&htim6); //Timer to count to 10 microseconds
/* USER CODE END 2 */
```

Further, add the following function definition in `main.c` in between the **`/* USER CODE BEGIN 4 */`** and **`/* USER CODE END 4 */`** blocks.

```
/* USER CODE BEGIN 4 */

void delay_uS(uint16_t delay)
{
    __HAL_TIM_SET_COUNTER(&htim6,0);
    while(__HAL_TIM_GET_COUNTER(&htim6) < delay)
    {

    }
}

/* USER CODE END 4 */
```

The function `delay_uS()` initially sets the count of timer TIM6 to zero upon entry using the timer macro `__HAL_TIM_SET_COUNTER`, and then waits for the timer count to reach the value assigned to the 16-bit unsigned integer `delay` provided as input argument. Within the `while` loop, the timer count is continuously checked by utilizing the timer macro `__HAL_TIM_GET_COUNTER`. Once the timer count has reached the value of the input argument `delay`, the function will return.

Add the function prototype for `delay_uS()` in `main.c` in between the **`/* USER CODE BEGIN PFP */`** and **`/* USER CODE END PFP */`** blocks as shown in the image below.

```
/* USER CODE BEGIN PFP */
void delay_uS(uint16_t delay);
/* USER CODE END PFP */
```

**Step 4:** Add the following function definition on the next page in main.c in between the **`/* USER CODE BEGIN 4 */`** and **`/* USER CODE END 4 */`** blocks.

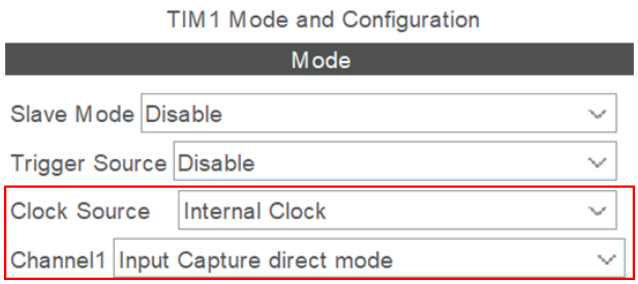
```
void HCSR04_Read()
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
    delay_uS(10);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
}
```

The function `HCSR04_Read()` sets the Trig pin of the ultrasonic sensor (connected to Pin PA6 of the microcontroller) to a 'Logic 1' for a period of 10 microseconds in order to start the distance measurement. This is performed by setting and resetting the Trig Pin of the ultrasonic sensor immediately before and after performing a call to the delay_uS() function, respectively, with an input argument of 10.

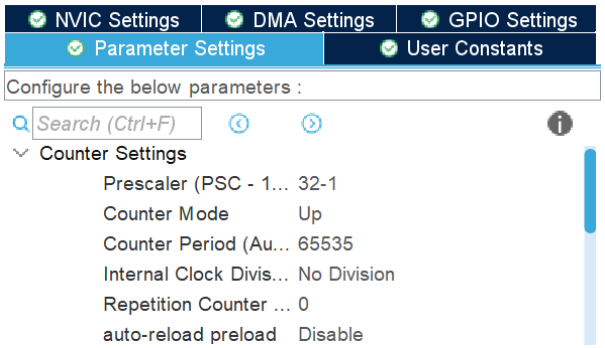
Add the function prototype for `HCSR04_Read()` in `main.c` in between the **`/* USER CODE BEGIN PFP */`** and **`/* USER CODE END PFP */`** blocks as shown in the image below.

```
/* USER CODE BEGIN PFP */
void delay_uS(uint16_t delay);
void HCSR04_Read();
/* USER CODE END PFP */
```

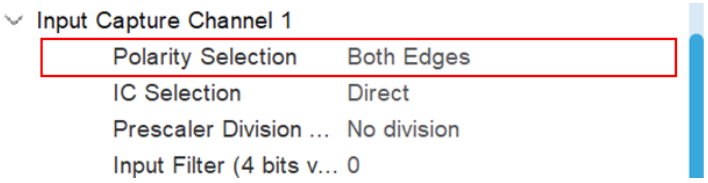
**Step 5:** Activate and configure timer TIM1 under the .ioc file of the project such that the timer can be utilized to measure the width of the pulse generated on the Echo pin of the sensor. This will be performed by **capturing the count of the timer at both the rising and falling edge of the signal from the Echo pin**. In order to allow the timer to capture the signal from the Echo pin (connected to Pin PA8 on the microcontroller), **set Pin_A8 to TIM1_CH1 on the Pinout View** and under the "TIM1 mode and configuration settings", select the clock source and Channel 1 options as shown in the image on the next page.



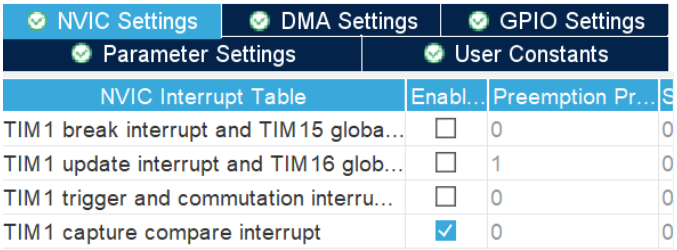

The counter settings for TIM1 should be chosen as shown in the image below (assuming APB2 clock frequency of 32 MHz). Based on this setting, the count of the timer increments at a rate of 1 per microsecond (similar to TIM6 in the project).



Since we are planning to capture the count of the timer TIM1 at both the rising and falling edges of the Echo pin, select “Both Edges” for the “Polarity Selection” option under Input Capture Channel 1 settings for timer TIM1 as shown in the image below..




In addition, since we will utilize an interrupt callback function to record the count of timer TIM1 at both the rising-edge and falling-edge of the Echo pin, check the “TIM1 capture compare interrupt” option under “NVIC Settings” of the timer.




**Step 6:** Save the .ioc file and generate code. In the file named `main.c`, add the following code in between the `/* USER CODE BEGIN 2 */` and `/* USER CODE END 2 */` blocks to start the timer TIM1 under interrupt capture mode.

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start(&htim6); //Timer to count to 10 microseconds
HAL_TIM_IC_Start_IT(&htim1, TIM_CHANNEL_1); //Timer to measure distance
/* USER CODE END 2 */
```

Further, add the definition for the timer interrupt callback function **`HAL_TIM_IC_CaptureCallback`**`()` in `main.c` in between the `/* USER CODE BEGIN 4 */` and `/* USER CODE END 4 */` blocks as shown in the image below.

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if ((htim == &htim1) && (htim->Channel == 1))
    {
        if (flag == 0)
        {
            count_1 = HAL_TIM_ReadCapturedValue(&htim1, TIM_CHANNEL_1);
            flag = 1;
        }
        else
        {
            count_2 = HAL_TIM_ReadCapturedValue(&htim1, TIM_CHANNEL_1);
            __HAL_TIM_SET_COUNTER(htim,0);
            distance = (count_2-count_1)/58.0;
            flag = 0;
        }
    }
}
```

The code above uses four variables `flag`, `count_1`, `count_2` and `distance` (in cm) in order to perform the distance calculation. Declare these variables in between the **`/* USER CODE BEGIN PV */`** and **`/* USER CODE END PV */`** blocks in the `main.c` file as shown in the image below.

```
/* USER CODE BEGIN PV */
int flag = 0;
float distance = 0;
uint16_t count_1 = 0;
uint16_t count_2 = 0;
/* USER CODE END PV */
```

The **`HAL_TIM_IC_CaptureCallback()`** timer interrupt callback function (on the previous page) is triggered every time a rising-edge or falling-edge is detected on a signal connected to any of the input channels of a timer on the microcontroller. In the present case, a call to this function will be triggered whenever a rising-edge or falling-edge is detected on Channel 1 of Timer TIM1, which is connected to the Echo pin of the ultrasonic sensor.

The first `if-statement` in the interrupt callback function checks whether Channel 1 of Timer TIM1 had triggered the function call. If so, the function then checks, using an if-else statement, whether a rising-edge or falling-edge on the signal from the Echo pin had triggered the function call by inspecting the value of the global variable `flag` (initialized as zero).

- If `flag == 0` (rising-edge), the captured count value of Timer TM1 is obtained using the function **`HAL_TIM_ReadCapturedValue()`** and then assigned to the variable `count_1`. Further, `flag` is now set to 1 in order to ensure the next entry into the function is recognized as one triggered by the subsequent falling-edge of the signal on the Echo pin.
- If `flag == 1` (falling-edge), the captured count value of Timer TIM1 is again obtained using the function **`HAL_TIM_ReadCapturedValue()`**, but now assigned to the variable `count_2`. Further, the distance from the nearest obstacle is calculated (in cm), after which `flag` is now reset to 0 in order to ensure the next entry into the function is recognized as one triggered by a new rising-edge of the signal on the Echo pin.

**Step 7:** In `main.c`, call the function `HCSR04_Read()` within the while(1) loop once every 500 milliseconds, as shown in the image below.

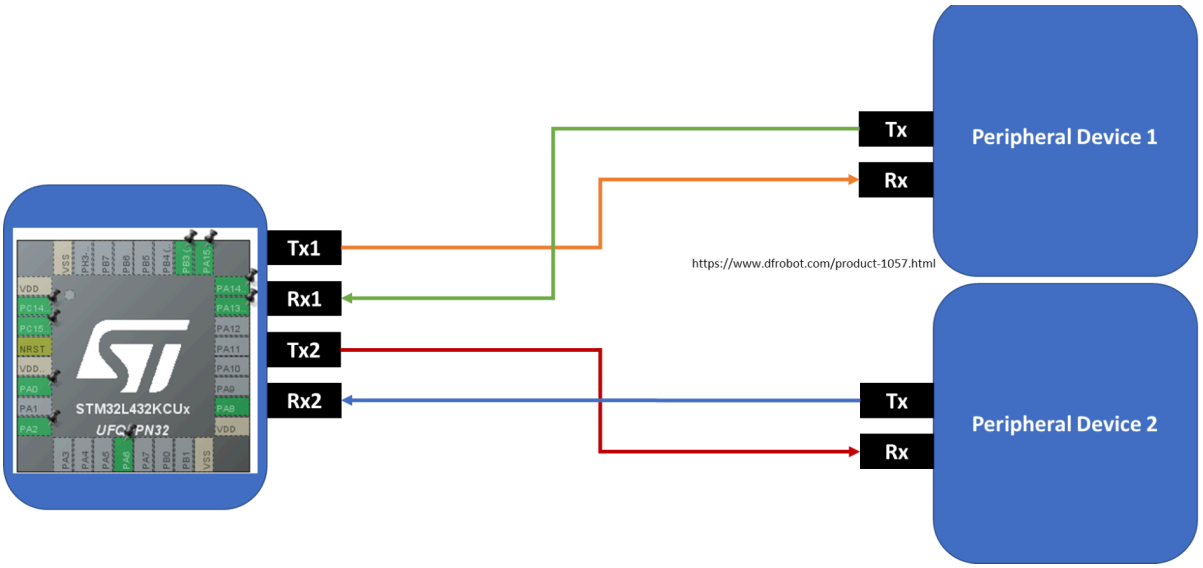
```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    HCSR04_Read();
    HAL_Delay(500);
  /* USER CODE END WHILE */
```

## Task 2(b): Transmitting the measured distances from the ultrasonic sensor to the computer using UART interface

Currently, the timer interrupt callback function **`HAL_TIM_IC_CaptureCallback()`** in main.c allows calculation of the distance measured by the ultrasonic sensor from the nearest obstacle. A convenient and rather effective means to debug the firmware of a microcontroller during runtime is to transmit the current value of variables in the code, when required, to a computer using serial communication. In the present case,

while the ultrasonic sensor is periodically measuring distance from the nearest obstacle, one way for us to verify the functionality of the code is by printing the value of the `distance` variable in the code, each time it's calculated, to a computer.

Serial communication is the process of sending/receiving data one bit at a time, one after the other in series. This type of communication is commonly used by computers, particularly over USB connections. One method of serial communication is UART (Universal asynchronous receiver transmitter), which refers to an interface that implements serial communication over a 2-wire bus: a transmission wire (labelled Tx) and a receiving wire (labelled Rx), to send and receive information between devices. For instance, the STM32 Nucleo-L432KC board supports upto two UART interfaces, which can be used to connect the board to two different peripheral devices that support UART communication in full-duplex mode as shown in the figure below.




In UART-based serial communication, data is transmitted from the Tx line of the sender to the Rx line of the receiver in the format of packets, the format of which is shown in the figure below.

| Start Bit (1 bit) | Data Frame (5 to 9 Data Bits) Assume 8 bits | Parity Bits (0 to 1 bit) Assume No Parity Check | Stop Bits (1 to 2 bits) Assume 1 bit |
|---|---|---|---|

Each UART packet is composed of 4 different fields:

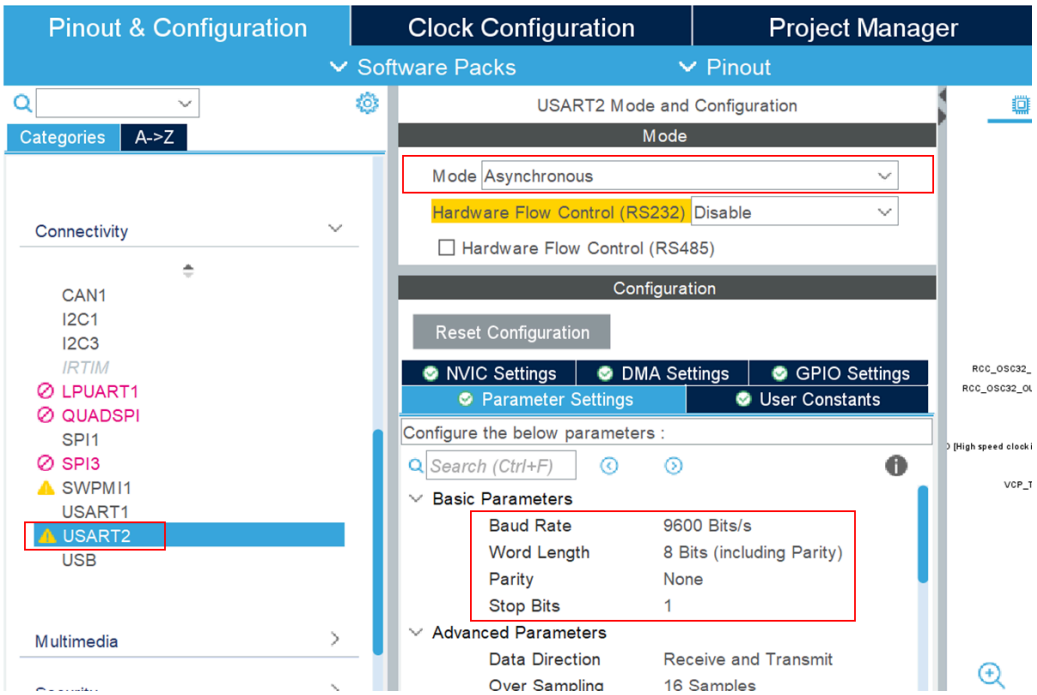
- The **Start Bit** allows the transmitter to signal the receiver that it is going to begin sending new data.
- The **Data Frame** can contain a value that can be represented within 5 to 9 bits, and this is essentially the data which the transmitter wants to send over to the receiver.
- The **Parity Bit** is used by the receiver to check for any errors introduced in the data during transmission.
- The **Stop Bits** allows the transmitter to signal the receiver that data transmission has been completed.

In order for two devices to communicate using the UART interface, they need to come to an agreement on the following:

- **A common UART packet format:** For instance, in this task we will ensure that both the STM32 Nucleo-L432KC board and the computer will agree with a packet format which contains **8** data bits, **N**o parity check and **1** Stop bit, which in-short is commonly known as the **8N1** UART configuration.
- **The same speed of communication (baud rate):** Since the transmitter and receiver does not share a common clock line during serial communication using UART), it is critical that the receiver is aware of the rate at which bits within the UART packet must be sampled in order to decode the data without errors. For this task, we will set a baud rate of 9600 bits per second for both the transmitter and receiver.

The default settings for the STM32 Nucleo-L432KC board in the IDE will auto-allocate one of the UART interfaces named UART2 for serial communication with the computer via the micro USB jack on the board. The pair of pins utilized for serial communication over UART2 are PA2 (labelled VCP_TX in the Pinout view) and PA15 (labelled VCP_RX in the Pinout view). Thus, VCP_TX and VCP_RX are the names assigned to the transmitter and receiver pins, respectively, on the STM32 board that allow communication between the board and the computer via the UART interface. You may also notice that these two pins (PA2 and PA15) are not allocated on the physical board, as they are instead connected with the micro USB jack on the STM32 board to facilitate serial communication using the UART interface.

**Step 8:** Connect the STM32 Nucleo-L432KC board to your computer using the USB Type A-to-Micro B cable provided in the kit. In the .ioc file of the project, click on "USART2", select mode as "Asynchronous" and then set the basic parameters of UART2 as shown in the image below. Save the .ioc file and then generate code.




**Step 9:** In the `main.c` file, include the `stdio.h` and `string.h` header files in between the `/* USER CODE BEGIN Includes */` and `/* USER CODE END Includes */` blocks as shown in the image on the next page.

```
/* Private includes ----------
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */
```

Declare a character array string_1 of size 40 in between the **/* USER CODE BEGIN PV */** and **/* USER CODE END PV */** blocks as shown in the image below.

```
/* USER CODE BEGIN PV */
int flag = 0;
float distance = 0;
uint16_t count_1 = 0;
uint16_t count_2 = 0;
char string_1[40];
/* USER CODE END PV */
```

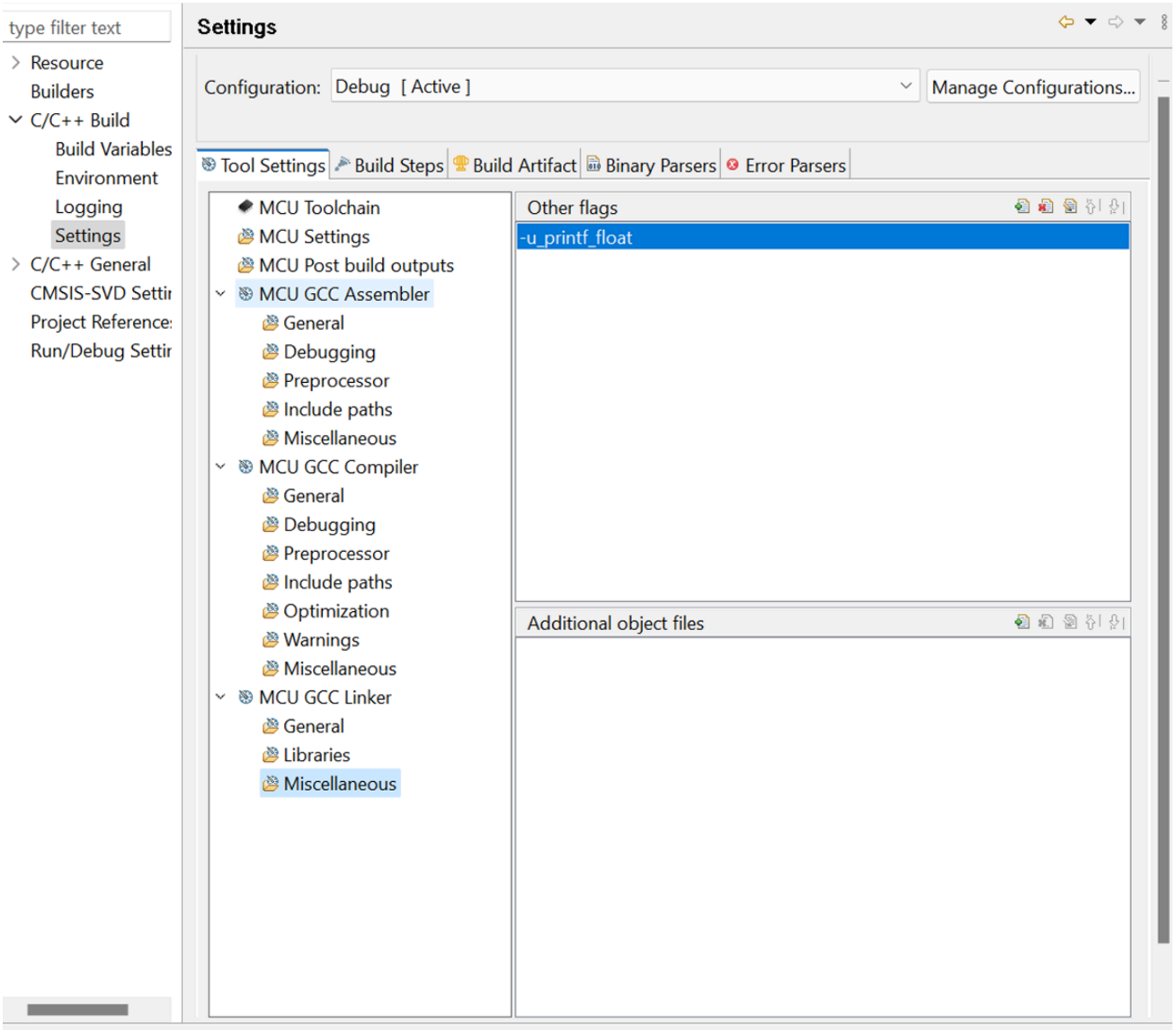
Add the following lines of code right after the distance is calculated within the else block in the **HAL_TIM_IC_CaptureCallback**() function in main.c.

```
sprintf(string_1,"distance: %f cm\n", distance);
HAL_UART_Transmit(&huart2, (uint8_t*)string_1, strlen(string_1),10);
```

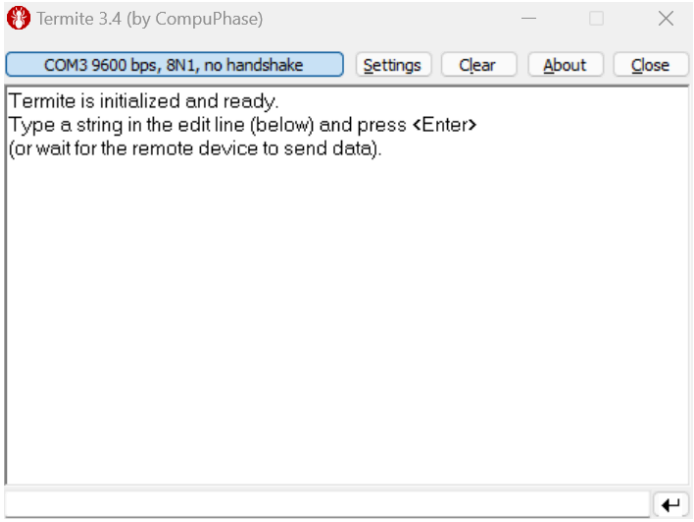
The ***HAL_UART_Transmit (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)*** API in the code above allows to transmit data from the sender to the receiver using the UART interface. The function takes in four input arguments:

- **huart:** UART handle, which specifies whether the communication occurs via the UART1 or UART2 interface on the STM32 board.
- **pData:** Pointer to data buffer which contains the data to be transmitted, which in this case is `string_1`.
- **Size:** The number of bytes to be transmitted, which in this case is the length of string_1 (excluding the terminating NULL character).
- **Timeout:** The maximum time in milliseconds during which the function should wait for the data to be available for transmission

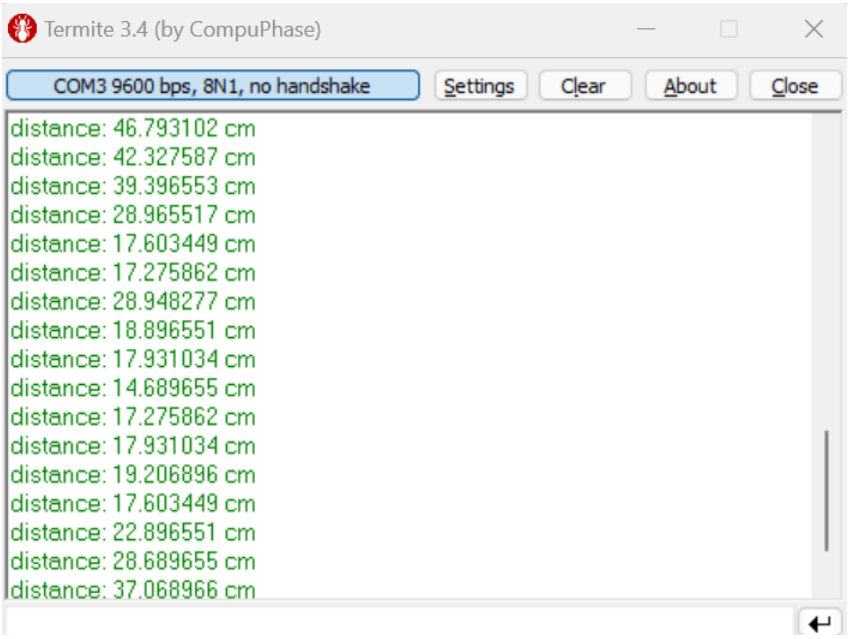
In order to handle floating point numbers on the STM32 microcontroller, click on Project → Properties on the toolbar of the IDE, and then add the `-u_printf_float` flag under C/C++ Build → MCU GCC Linker → Miscellaneous as shown in the image on the next page.


type filter text
Resource
Builders
C/C++ Build
Build Variables
Environment
Logging
Settings
C/C++ General
CMSIS-SVD Settir
Project Reference:
Run/Debug Settir
Settings
Configuration: Debug [ Active ]
Manage Configurations...
Tool Settings
Build Steps
Build Artifact
Binary Parsers
Error Parsers
MCU Toolchain
MCU Settings
MCU Post build outputs
MCU GCC Assembler
General
Debugging
Preprocessor
Include paths
Miscellaneous
MCU GCC Compiler
General
Debugging
Preprocessor
Include paths
Optimization
Warnings
Miscellaneous
MCU GCC Linker
General
Libraries
Miscellaneous
Other flags
-u_printf_float
Additional object files

**Step 10:** Open Termite RS-232 terminal on the computer. You could note that the default baudrate and UART packet format is set as 9600 bits per second and 8N1, respectively, as shown in the image below. If this is not the case, you may change the communication parameters by clicking on "Settings" on Termite. In addition, the COM port assigned for serial communication on the computer is COM3 as shown in the image on the next page. However, this setting could be different on your computer and if needed, you may set the correct COM port by clicking on "Settings" on the terminal. If an incorrect COM port is selected, the transmitted data will not appear on the terminal.




**Step 11:** Build the STM32 project on the IDE and then run as STM32 C/C++ application. The distance measured by the ultrasonic sensor from the nearest obstacle should now be printed continuously on the terminal as shown in the image below.




**Step 12:** Close the Termite software and create a Python script `read_serial.py` containing the code on the next page on your computer.

```python
import serial
ser = serial.Serial(port='COM3', baudrate=9600, bytesize=8,parity='N',stopbits=1,timeout=2)
print(ser.name)
while (1):
    distance = ser.readline()
    print(distance.decode('ascii'))
print("Communication timed out")
```

The activity conducted in each line of the Python script is described as follows:

**Line 1:** Imports the PySerial module
**Line 2:** Opens COM3 serial port on the computer to receive data at a **baudrate of 9600 bits per second** while complying with the **8N1 UART packet format**. In addition, the **timeout is set as 2 seconds** for the communication.
**Line 3:** Prints the name of the serial port to indicate that the script has managed to open the set serial port for communication
**Lines 4-6:** Continuously reads one line at a time from the serial port, decodes the bytes received as an ASCII string and then prints the distance onto the powershell
**Line 7:** Prints "Communication timed out" onto the powershell once the newline character ('\n') has not been received by the computer through the UART interface for a period of 2 seconds (timeout set for the communication)

On the powershell, navigate to the folder where read_serial.py has been saved, and then run the Python script using the following command.

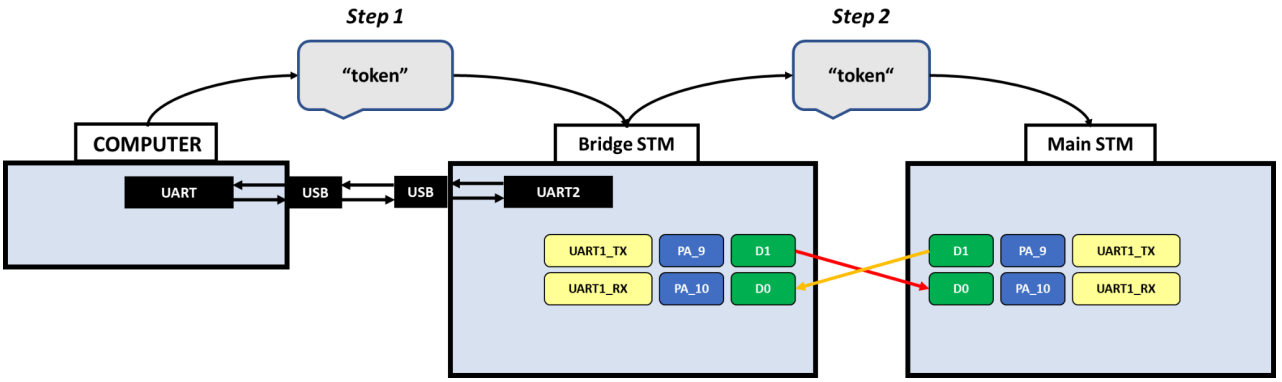
```
PS C:\code> python read_serial.py
```

This should print the distance measured by the ultrasonic sensor from the nearest obstacle onto the powershell, similar to what was previously observed on the Termite software.

```
PS C:\code> python read_serial.py
COM3
distance: 14.689655 cm

distance: 27.310345 cm

distance: 21.724138 cm

distance: 29.465517 cm

distance: 17.603449 cm

distance: 15.017241 cm

distance: 12.534483 cm

distance: 13.137931 cm

distance: 13.724138 cm

distance: 18.568966 cm

distance: 21.448277 cm

distance: 22.603449 cm

distance: 23.482759 cm

distance: 24.362068 cm
```

Modify the STM32 project and the Python script in order to incorporate the following functionalities towards the project specifications in the unit.

1. Declare a new integer variable `min_distance` in the main.c file of the STM32 project. This variable will now refer to the minimum distance (in cm) at which a user has to be detected by the ultrasonic sensor in order to start the audio recording. Since we will be covering ADCs during the Week 6 activity, for now you could indicate that the audio recording has started by lighting up the green user LED LD3 connected to pin PB3 on the microcontroller. Further, once the user is away from the ultrasonic sensor by a distance greater than the minimum distance set, you could indicate that the audio recording has stopped by turning OFF the green user LED. The `min_distance` variable can be initialized to a value of 10 in the code, which will refer to a distance of 10 cm from the nearest obstacle.
2. Instead of the existing polling with delay mechanism used to periodically trigger the ultrasonic sensor, utilize a new timer and the associated interrupt callback function in the STM32 project to periodically call the `HCSR04_Read()` function once every 500 milliseconds. The while(1) loop in the `main.c` file should not contain any code after this modification.
3. Add code to the Python script and the STM32 project such that the Python script can now act as a command line interface that allows the user to select the operating mode of the audio recording system. This will require the use of the API **HAL_UART_Receive()** on the STM32 microcontroller.
    - By default, the system should start in an idle state during which no recording is conducted (green user LED remains OFF).
    - If the string "m" (representing manual recording mode) is sent by the command line interface to the STM32 board, the recording should be conducted (during which the green user LED should be turned ON) indefinitely until the user requests a change of operating mode.
    - If the string "d" (representing distance trigger mode) is sent by the command line interface to the STM32 board, the recording should be conducted (green user LED be turned ON) while the ultrasonic sensor detects a user within a distance of 10 cm.
    - The operating mode should be switchable, based on user input through the command line interface, at any point of time while either mode is active.
4. Modify the STM32 project such that the microcontroller can now accept commands that have a fixed length of 8 characters ("manual  " or "distance") to switch the operating mode. This will require the use of the APIs **HAL_UART_Receive_IT()** and **HAL_UART_RxCpltCallback()** on the STM32 microcontroller in order to receive and process data using the UART2 interface under the interrupt mode. The **HAL_UART_Receive_IT()** API starts reception of data of a certain number of bytes (8 in this case) and the **HAL_UART_RxCpltCallback()** function is triggered once the required number of bytes have been received.
5. For your project milestone demo next week, you will need to transmit messages over UART from one STM2 board to another. As a starting step towards the project milestone, now use a new STM32 board (Bridge STM) that acts as a bridge in between the existing STM32 board (Main STM) and the computer as depicted in the figure on the next page. For this purpose, the Bridge STM and Main STM should be connected to each other using the TX and RX pins PA_9 and PA_10 as shown in the figure on the next page. The system should operate as follows:
    - By default, the system should start in an idle state where the Bridge STM awaits a five-character string ("token") from the command line interface via UART 2.
    - Upon reception of the string, the Bridge STM should relay the string to the Main STM via UART1.
    - Once the Main STM receives the string, it should indicate this activity by switching the state of the onboard green user LED.




For the list of APIs on both STM32 and the PySerial module required to perform these tasks, pls refer to the following resources:

1. STM32 L4/L4+ Hardware Abstraction Layer and Low-Layer Drivers
2. PySerial documentation